Tervezési minták I.
1.rész
Lengyel forma és kiértékelése

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

Feladat

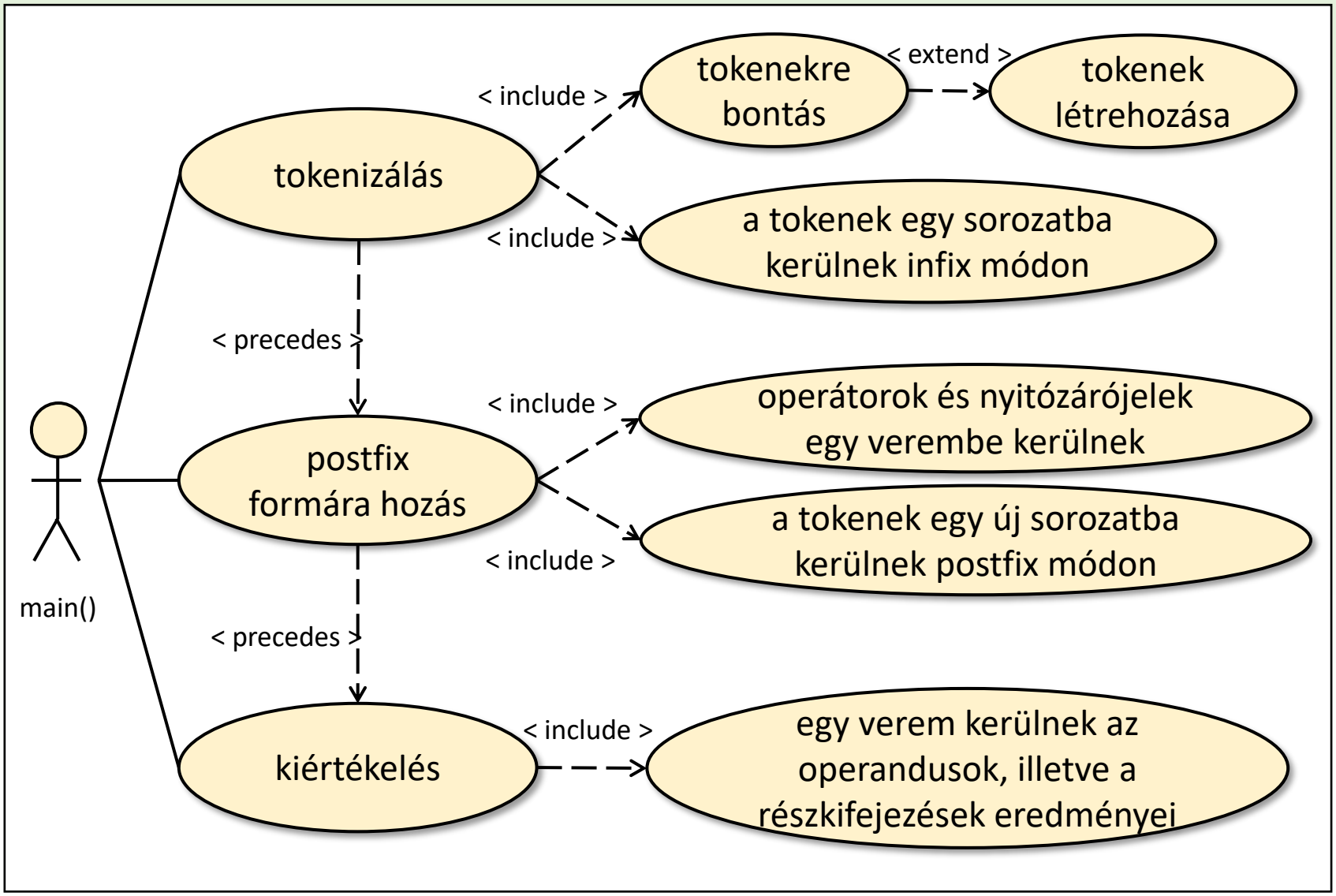
Alakítsunk át egy infix formájú aritmetikai kifejezést postfix formájúra (RPN), és számoljuk ki az értékét.

infix forma		postfix forma		érték
5+(11+26)*(43-4)	→	**5 11 26 + 43 4 - * +**	→	**1448**

Az átalakításhoz is, és a kiértékeléshez is egy-egy vermet szoktak használni. Az elsőben műveleti jeleket és a nyitózárójeleket helyezünk el az átalakítás során, a másodikban operandusokat, illetve a részeredményeket.

Szükség lesz arra, hogy a kifejezéseinkben egymástól elválasztva tudjuk kezelni a lexikális egységeket (tokeneket), azaz a műveleti jeleket, zárójeleket, számokat.

Elemzés


tokenekre bontás
< extend >
tokenek létrehozása
< include >
tokenizálás
< include >
a tokenek egy sorozatba kerülnek infix módon
< precedes >
< include >
operátorok és nyitózárójelek egy verembe kerülnek
postfix formára hozás
< include >
a tokenek egy új sorozatba kerülnek postfix módon
main()
< precedes >
< include >
kiértékelés
egy verem kerülnek az operandusok, illetve a részkifejezések eredményei

Objektumok

tokenek: operátorok (Operator), operandusok (Operand),
zárójelek (LeftP, RightP) mind a Token osztály leszármazottjai

sorozatok: tokenek (hivatokzásaik) gyűjteményei (List<Token>):
az infix formájú tokenizált kifejezést tárolja (infix)
a postfix formára hozott tokenizált kifejezést tárolja (postfix)

vermek: tokeneket (hivatokzásaikat) tároló verem (Stack<Token>),
egész számokat tároló verem (Stack<int>)

sztring: az infix formájú kifejezés a szabványos bemeneten (string)

Főprogram

```
int main() {
	char ch;
	do {
		cout << "Give me an arithmetic expression:\n";
		try{
		// Tokenization
			vector<Token*> infix;
			...
		// Transforming into RPN
			vector<Token*> postfix;
			Stack<Token*> stackToken
			...
		// Evaluation
			Stack<int> stackInt;
			...
		} catch(Interrupt ex) { }

		deallocateToken(infix);

		cout << "\nDo you continue? Y/N";
		cin >> ch;
	} while( ch!='n' && ch!='N' );
	return 0;
}
```

itt példányosodnak majd a tokenek

Szintaktikai vagy szemantikai hiba miatt bekövetkezhet megszakítás.

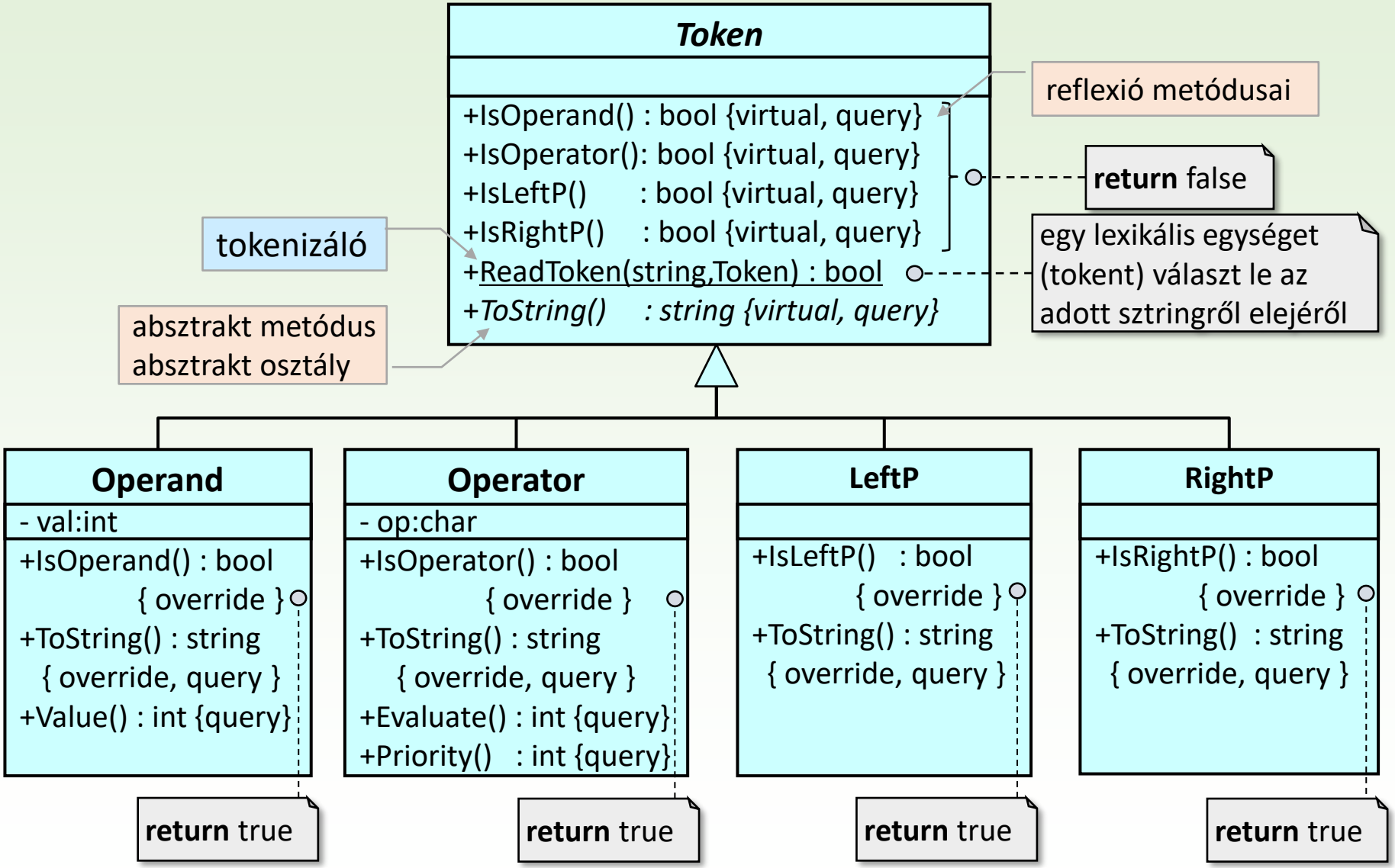
```
enum Interrupt { Error };
```

```
void deallocateToken(vector<Token*> & infix)
{
	for( Token* t : infix ) delete t;
}
```

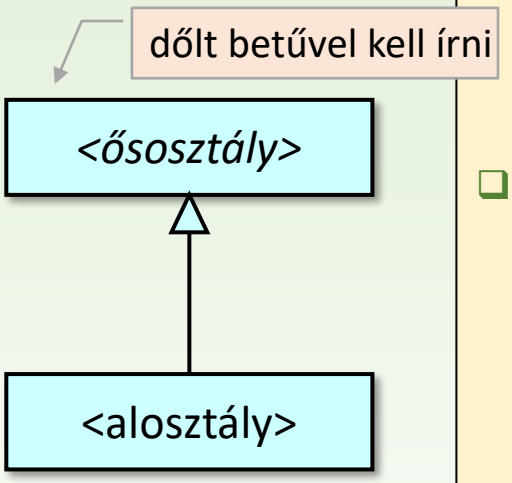
felszabadítja a tokenek helyfoglalásait

main.cpp

Token osztály és leszármazottjai

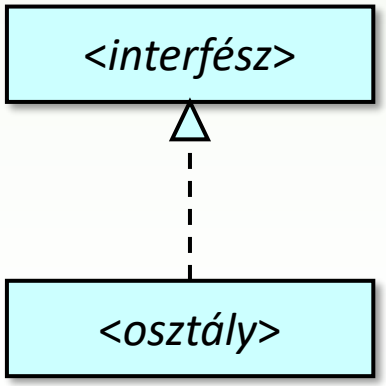


Absztrakt osztály, interfész



- Absztrakt (*abstract*) osztály az, amelyből nem példányosítunk objektumokat, kizárólag ősosztályként szolgálnak a származtatásokhoz.
- Egy osztály attól lesz absztrakt, hogy
  - konstruktorai nem publikusak,
  - vagy legalább egy metódusa absztrakt (dőlt betűvel kell írni), azaz nincs implementálva, csak származtatás során írjuk majd felül

- Interfésznek, azaz tisztán absztrakt (*pure abstract*) osztálynak nevezzük azt az osztályt, amelyiknek egyetlen metódusa sincs implementálva.
- Egy interfészből származtatott osztály, amelyik az interfész minden absztrakt metódusát implementálja, az megvalósítja az interfészt.



Token osztály

```
class Token
{
public:
    class IllegalElementException : public std::exception {
    private:
        char _ch;
    public:
        IllegalElementException(char c) : _ch(c){}
        const char* what() const noexcept override {
            char* msg = new char[2]; msg[0] = _ch; msg[1] = '\0';
            return msg;
        }
    };
    virtual ~Token() {};
    virtual bool is_LeftP()      const { return false; }
    virtual bool is_RightP()     const { return false; }
    virtual bool is_Operand()    const { return false; }
    virtual bool is_Operator()   const { return false; }
    virtual bool is_End()        const { return false; }

    virtual std::string to_String() const = 0;

};
```

Beágyazott osztályként definiál egy kivételt, amelyet majd a kiváltásakor példányosítunk.
#include <exception>

Miért legyen virtuális a destruktor?

specifikációban még nem szerepelt

emiatt lesz absztrakt az osztály

token.h

Operand osztály

```
class Operand: public Token
{
private:
	int _val;
public:
	Operand(int v) : _val(v) {}

	bool is_Operand() const override { return true; }

	std::string to_String()  const override {
		std::ostringstream ss;
		ss << _val;
		return ss.str();
	}
	int value() const { return _val; }
};
```

konverzió

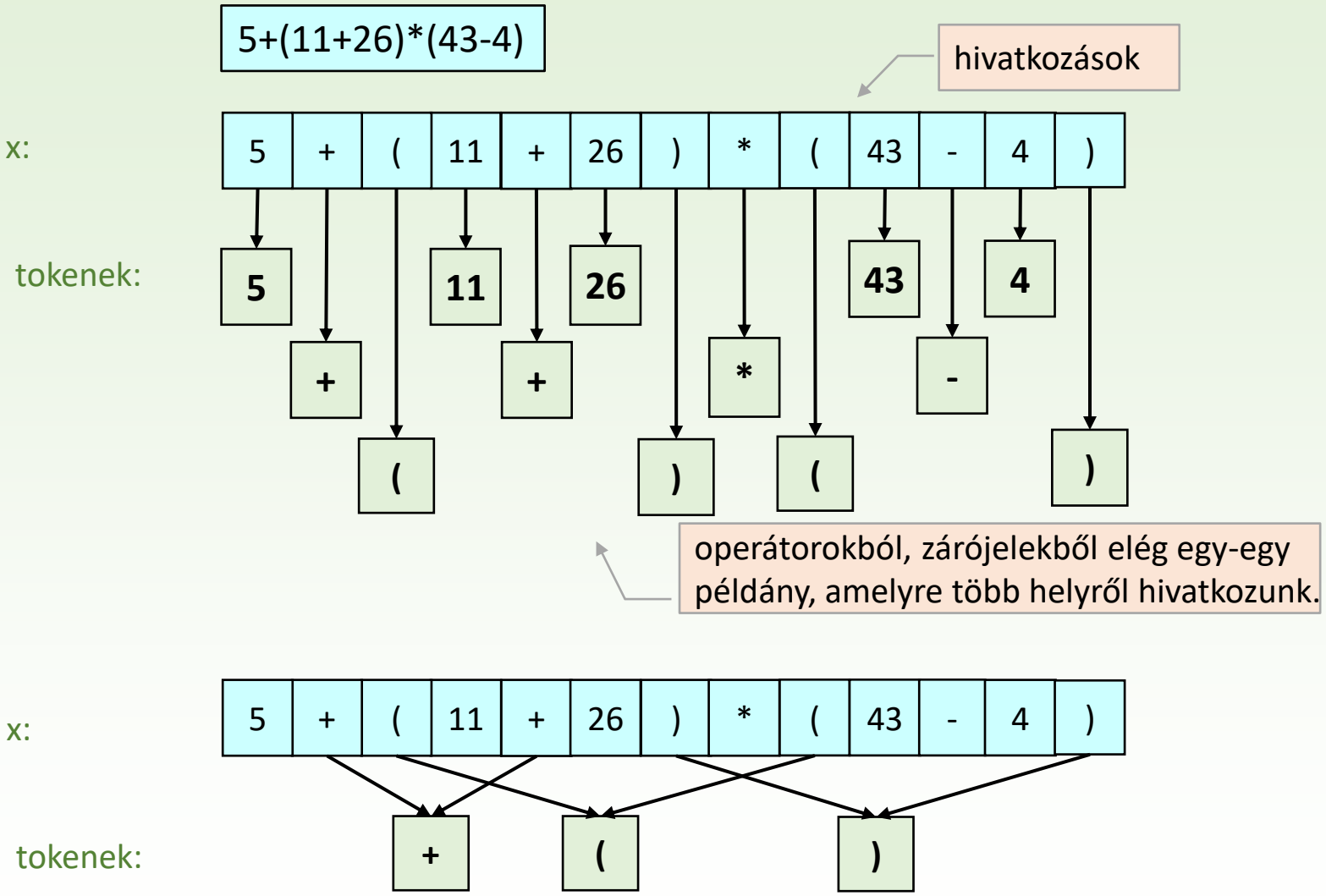
token.h

LeftP osztály

```
class LeftP : public Token
{
public:
	LeftP(){}
	bool is_LeftP()		const override { return true; }
	std::string to_String()	const override { return "("; }
};
```

token.h

Mielőtt tovább lépnénk



LeftP objektumból elég egy példány

```
class LeftP : public Token
{
private:
        LeftP(){}

        static LeftP *_instance;
public:
        static LeftP *instance() {
                if ( _instance == nullptr ) _instance = new LeftP();
                return _instance;
        }

        bool is_LeftP()             const override { return true; }
        std::string to_String()     const override { return "("; }
};
```

kívülről nem példányosítható, mert a konstruktor privát

egyetlen példányra hivatkozik, ha azt már létrehoztuk

itt már hívható a privát konstruktor

kívülről ezt a gyártó függvényt hívjuk a konstruktor helyett

token.h

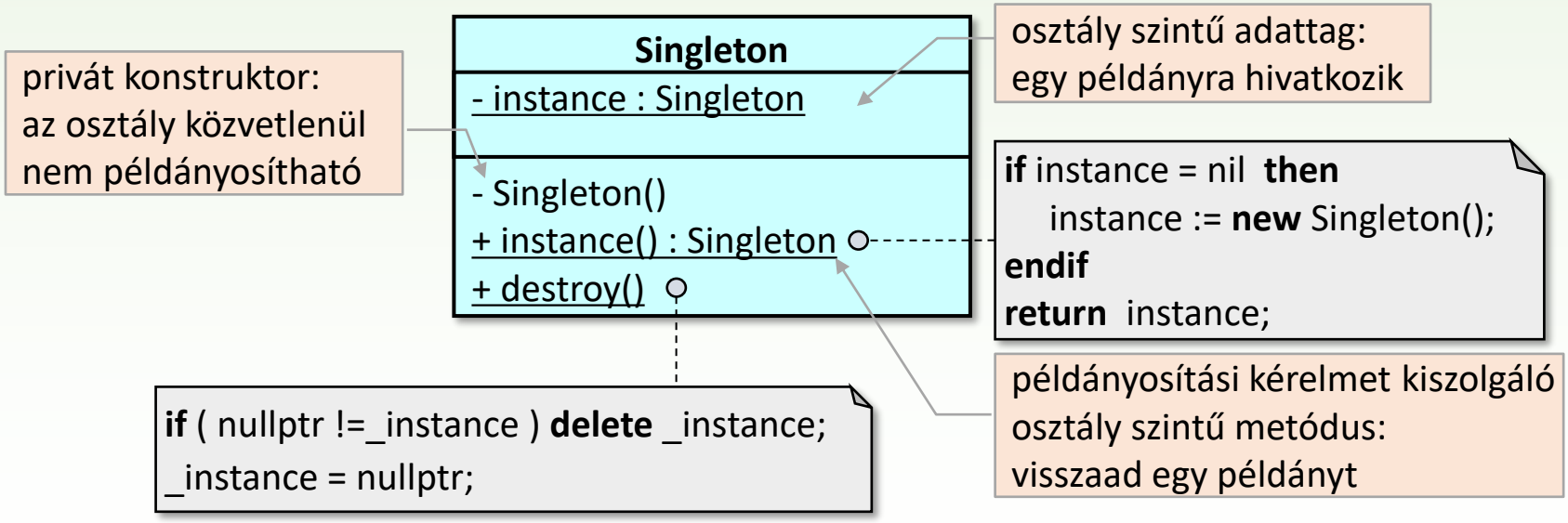
RightP, End osztályok ugyanígy

```
LeftP   *LeftP::_instance = nullptr;
```

token.cpp

Egyke (singleton) tervezési minta

- Egy osztályhoz legfeljebb egy objektumot kell példányosítani függetlenül a példányosítási kérelmek számától.



Programozási minták

- A programozási feladatok megoldási folyamata gyorsabb, az előállított program biztonságosabb, ha a megoldást korábbi, hasonló feladatok megoldásainál bevált minták alapján állítjuk elő.
- A szoftvertechnológiában a különféle programozási mintáknak számos csoportja jött létre. Ilyenek
  - az algoritmus minták,
  - a szoftver-architektúra minták (pl. modell-nézet-vezérlő minta),
  - a tervezési minták (tervminták, design patterns).
- A tervezési minták az objektumelvű modellezést támogató olyan osztálydiagram minták, amelyeket a programozási feladatok megoldásának tervezése során alkalmazunk azért, hogy a létrehozott program újrafelhasználható, rugalmasan módosítható, biztonságosan működő, és hatékony legyen.

Operator osztály (első próbálkozás)

```
class Operator: public Token {
private:
	char _op;
public:
	Operand(char v) : _op(v) {}
	bool is_Operator() const override { return true; }
	int Operator::evaluate(int leftValue, int rightValue) const {
		switch(_op){
			case '+': return leftValue+rightValue;
			case '-': return leftValue-rightValue;
			case '*': return leftValue*rightValue;
			case '/': return leftValue/rightValue;
			default: return 0;
		}
	}
	int Operator::priority() const {
		switch(_op){
			case '+': case '-': return 1;
			case '*' : case '/': return 2;
			default: return 3;
		}
	}
	std::string Operator::to_String()  const override {
		std::string ret; ret = _op; return ret;
	}
	bool Operator::is_Operator() const override { return true; }
};
```

el kellene érni, hogy az egyes operátorból csak egy példány jöjjön létre

ez a kód nem felel meg az Open-Close elvnek

Single responsibility

Open-Close

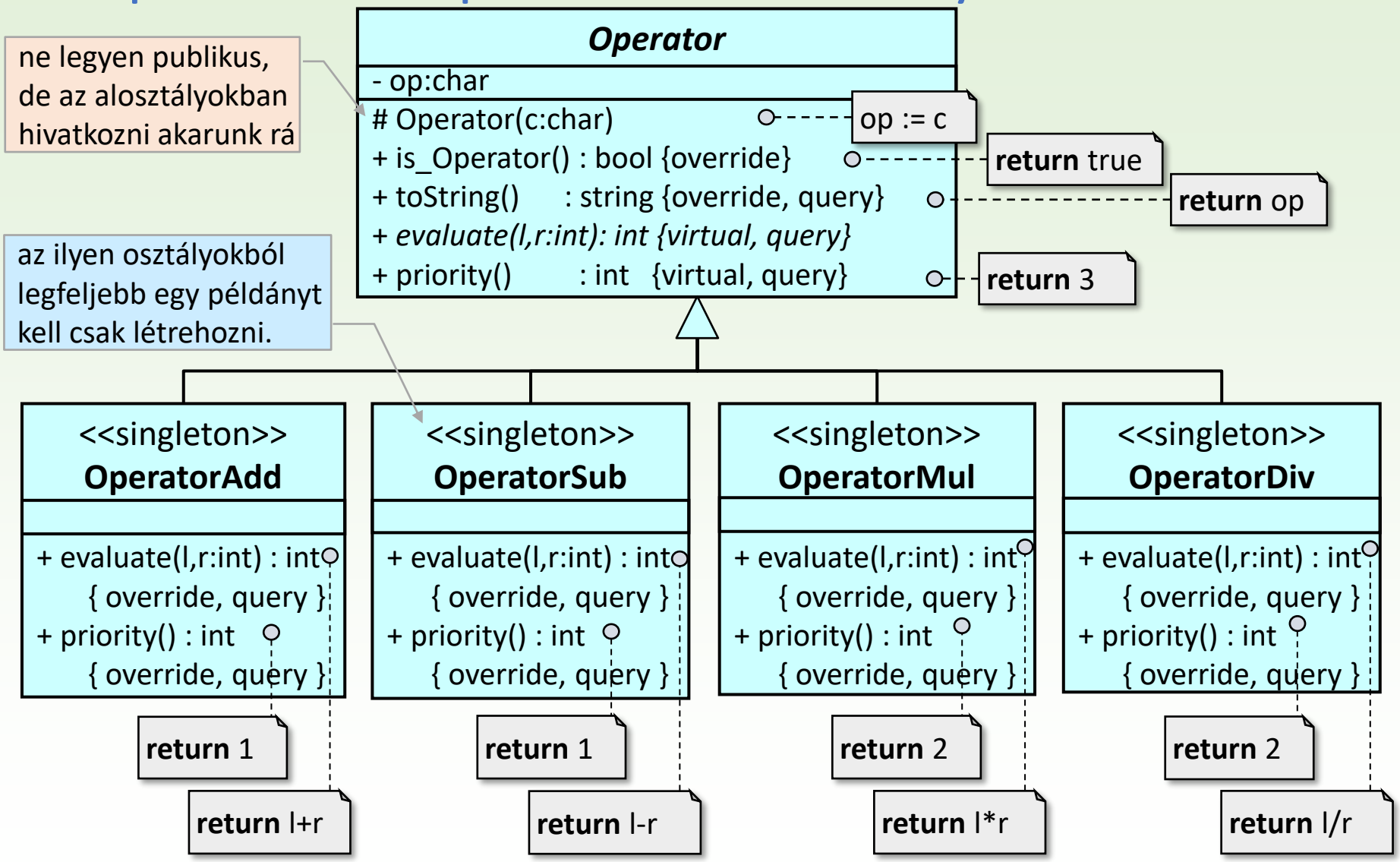
Liskov's substitution

I

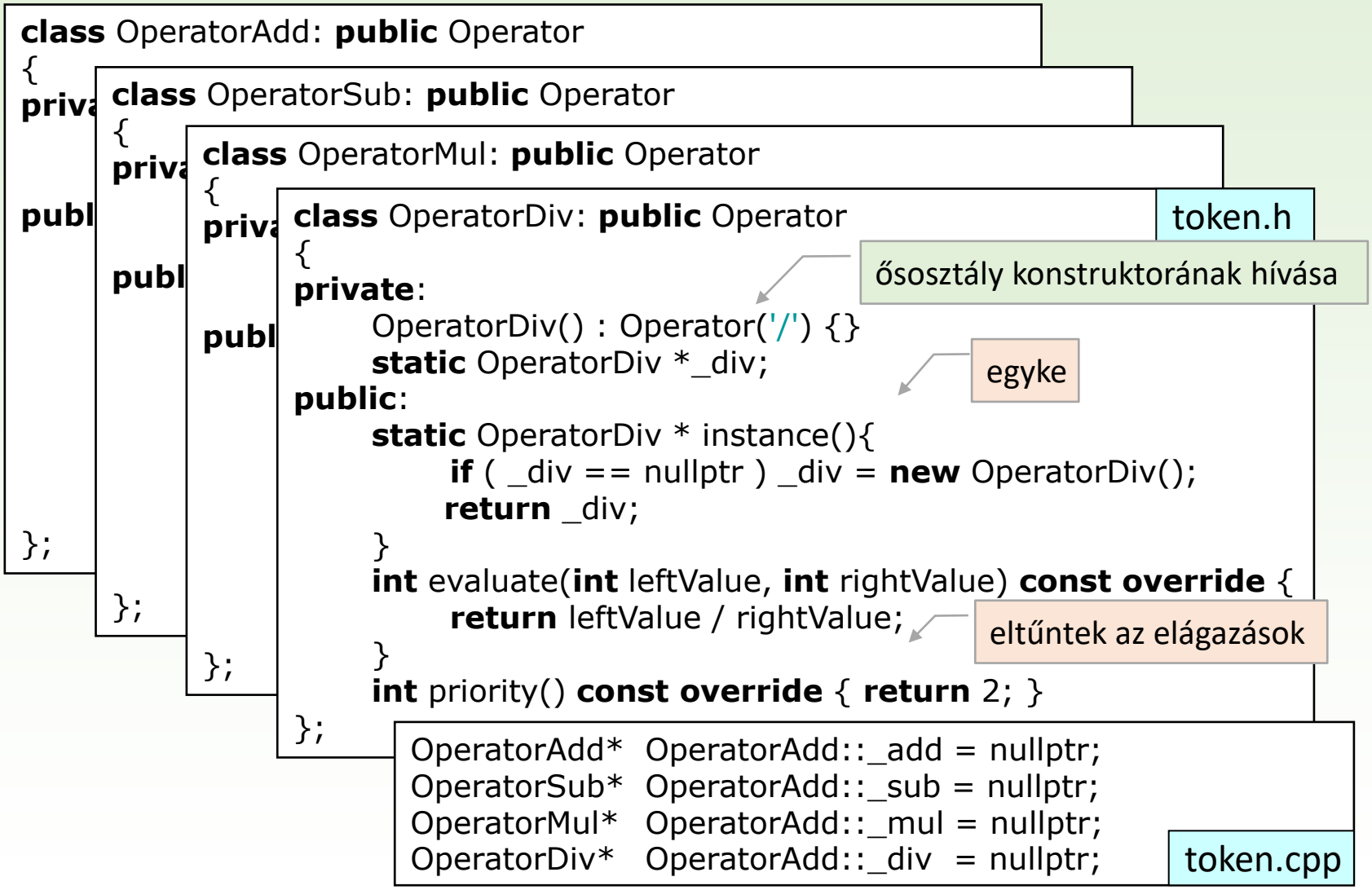
D

token.cpp

Speciális operátor osztályok



Egyke operátor osztályok



Tokenizálást végző operátor

```
istream& operator>> (istream &s, Token* &token){
        char ch;
        s >> ch;
        switch(ch){
                case '0'   : case '1' : case '2' : case '3' : case '4':
                case '5'   : case '6' : case '7' : case '8' : case '9':
                             s.putback(ch);
                             int intval;
                             s >> intval;
                             token = new Operand(intval); break;
                case '+'   : token = OperatorAdd::instance(); break;
                case '-'   : token = OperatorSub::instance(); break;
                case '*'   : token = OperatorMul::instance();  break;
                case '/'   : token = OperatorDiv::instance();  break;
                case '('   : token = LeftP::instance();    break;
                case ')'   : token = RightP::instance();   break;
                case ';'   : token = End::instance();      break;
                default:   if(!s.fail()) throw new Token::IllegalElementException(ch);
        }
        return s;
}
```

vissza a pufferbe

egykék használata

a bemenetként megadott aritmetikai kifejezést pontosvessző zárja le

token.cpp

Kifejezés tokenizálása

```
vector<Token*> infix;
// Tokenization
try{
        Token *token;
        cin >> token;
        while( !token->is_End() ){
                infix.push_back(t);
                cin >> token;
        }
}catch(Token::IllegalElementException *ex){
        cout << "Illegal character: " << ex->message() << endl;
        delete ex;
        throw Error;
}

if(0==infix.size()) {
        cout << "Empty expression\n";
        throw Error;
}
```

tokent olvas

az olvasás dobja

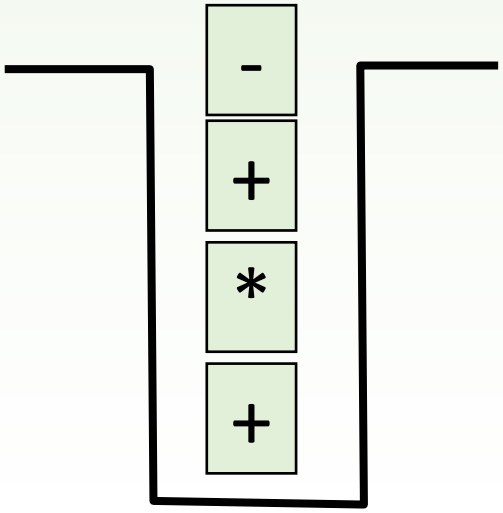
további műveletet nem végzünk

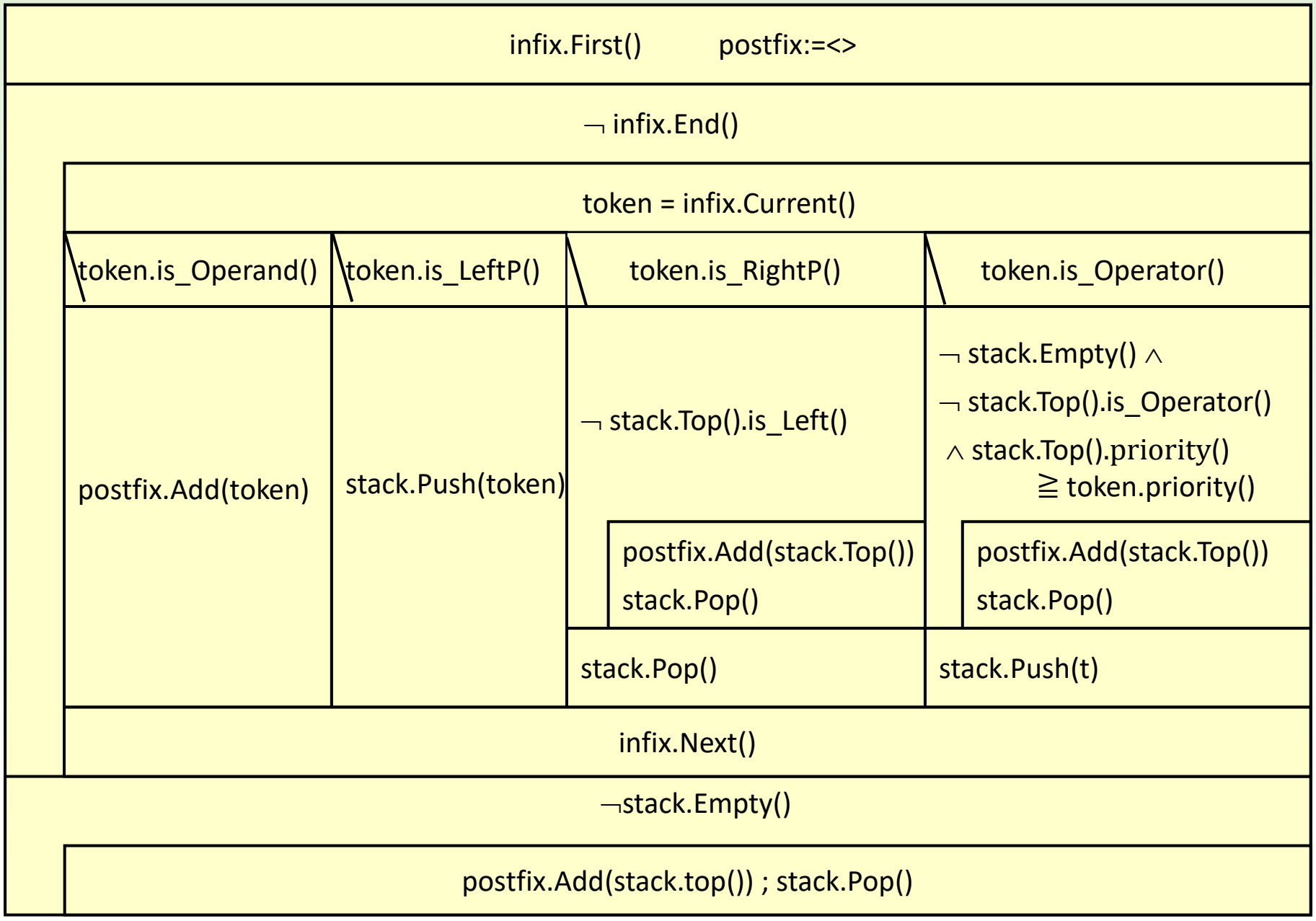
main.cpp

Infix formából postfix forma

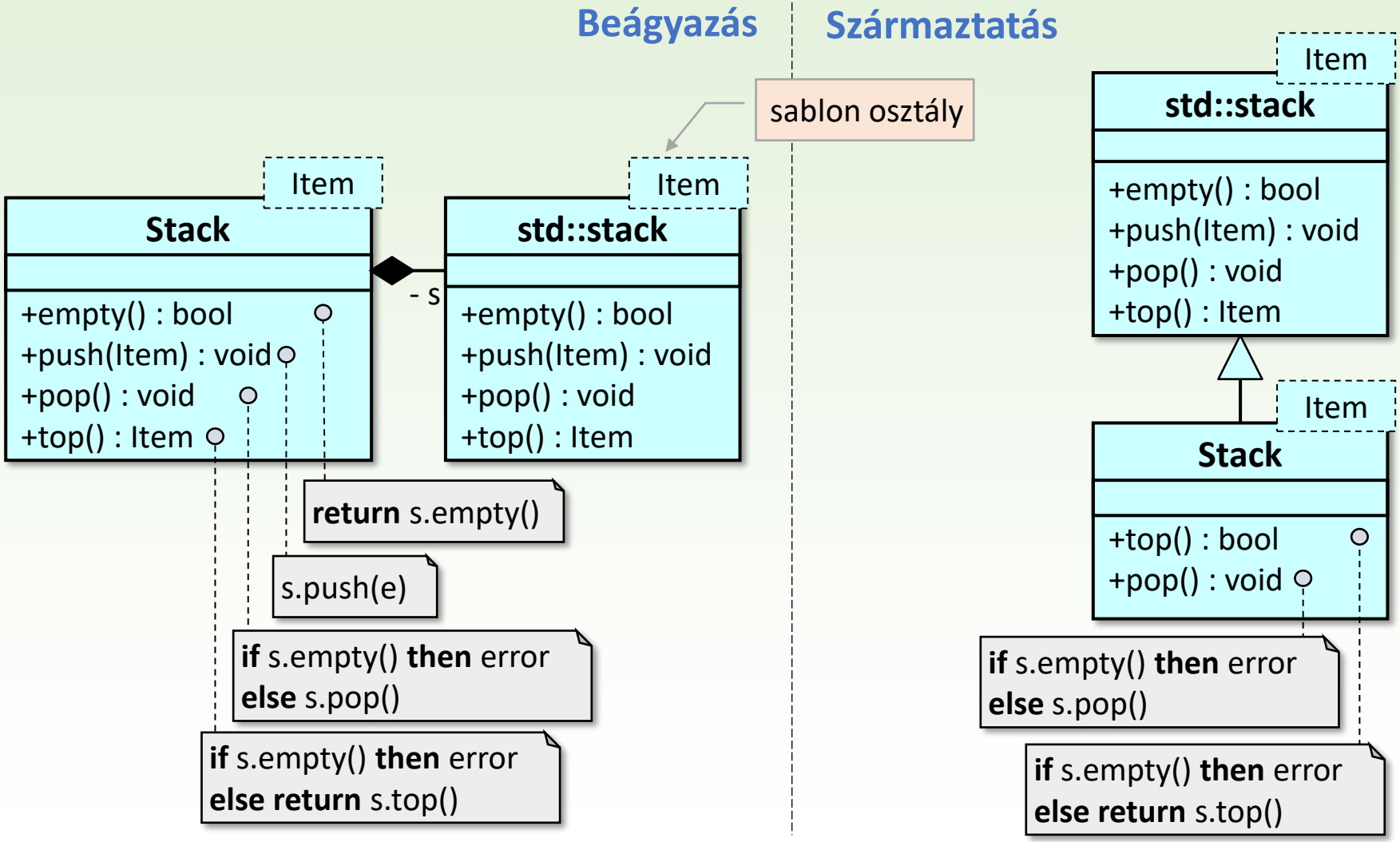
A bemenő sorozat nyitó zárójeleit és műveleti jeleit egy verembe tesszük (az alacsonyabb precedenciájú műveleti jel mindig helyet cserél az alatta levő magasabb precedenciájúval), minden más jelet közvetlenül a kimenő sorozatba másolunk. Csukó zárójel olvasása esetén illetve a bemenő sorozat feldolgozásának végén kiürítjük a verem tartalmát a leg(f)első nyitózárójeléig a kimenő sorozatba.

5	+	(	11	+	26	)	*	(	43	-	4	)




infix.First()
postfix:=<>
¬ infix.End()
token = infix.Current()
token.is_Operand()
token.is_LeftP()
token.is_RightP()
token.is_Operator()
postfix.Add(token)
stack.Push(token)
¬ stack.Top().is_Left()
postfix.Add(stack.Top())
stack.Pop()
stack.Pop()
¬ stack.Empty() ∧
¬ stack.Top().is_Operator()
∧ stack.Top().priority()
≧ token.priority()
postfix.Add(stack.Top())
stack.Pop()
stack.Push(t)
infix.Next()
¬stack.Empty()
postfix.Add(stack.top()) ; stack.Pop()

Alternatívák kivételt dobó veremsablonra

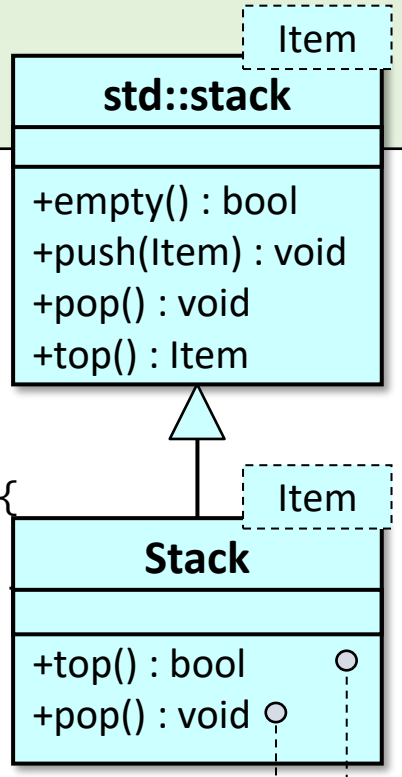


Kivételt dobó veremsablon

sablon osztály
típust helyettesítő paraméterrel

```
#include <stack>

template <typename Item>
class Stack : public std::stack<Item>
{
public:
	class EmptyStackException : public std::exception{
	public:
		const char* what() const noexcept override {
			return "empty stack";
		}
	};
	void pop() {
		if(std::stack<Item>::empty() ) throw new EmptyStackException;
		std::stack<Item>::pop();
	}
	Item top() const {
		if(std::stack<Item>::empty() ) throw new EmptyStackException;
		return std::stack<Item>::top();
	}
};
```



if s.empty() **then** error
else s.pop()

if s.empty() **then** error
else return s.top()

stack.hpp

Postfix formára hozás kódja

```
// Transforming into RPN

vector<Token*> postfix;
Stack<Token*> stackToken;

for( Token* token : postfix ){
        if      ( …
        else if ( …
        else if ( …
        else if ( …
}
while( ! stackToken.empty() ){
        if(stackToken.top()->is_LeftP() ){
                cout << "Syntax error!\n";
                throw Error;
        }else{
                postfix.push_back(stackToken
                stackToken.pop();
        }
}
```

sablon osztály példányosítása

felsorolás

a négy-ágú elágazás a következő dián

hiba lehetőség:
több nyitó zárójel, mint csukó

main.cpp

Postfix formára hozás ciklusmagja

```
if      ( token->is_Operand() ) postfix.Add(token);
else if (token->is_LeftP() )      stackToken.push(token);
else if (token->is_RightP() ){
      try{
            while( ! stackToken.top()->is_LeftP() ) {
                  postfix.push_back(stackToken.top());
                  stackToken.pop();
            }
            stackToken.pop();
      }catch(StackExceptions ex){
            if(ex==EMPTYSTACK){
                  cout << "Syntax error!\n";
                  throw Error;
            }
      }
}else if (token->is_Operator() ) {
      while( ! stackToken.empty() && stackToken.top()->is_Operator() &&
      ((Operator*) stackToken.top())->priority()>=((Operator*)t)->priority() ) {

            postfix.push_back(stackToken.top());
            stackToken.pop();
      }
      stackToken.push(token);
}
```

hiba lehetőség: több a csukó zárójel, mint nyitó

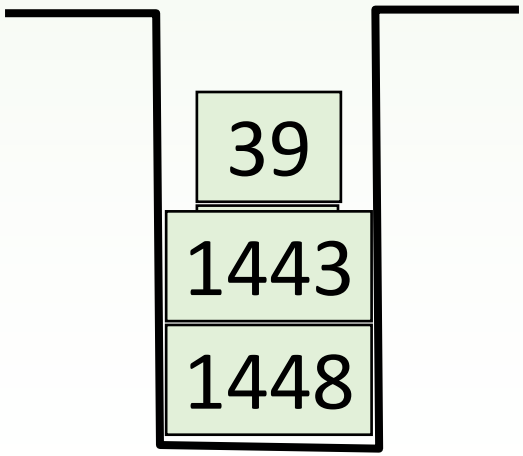
„static casting”: stackToken.top()->priority() nem jó, mert Token-ben nincs priority()

main.cpp

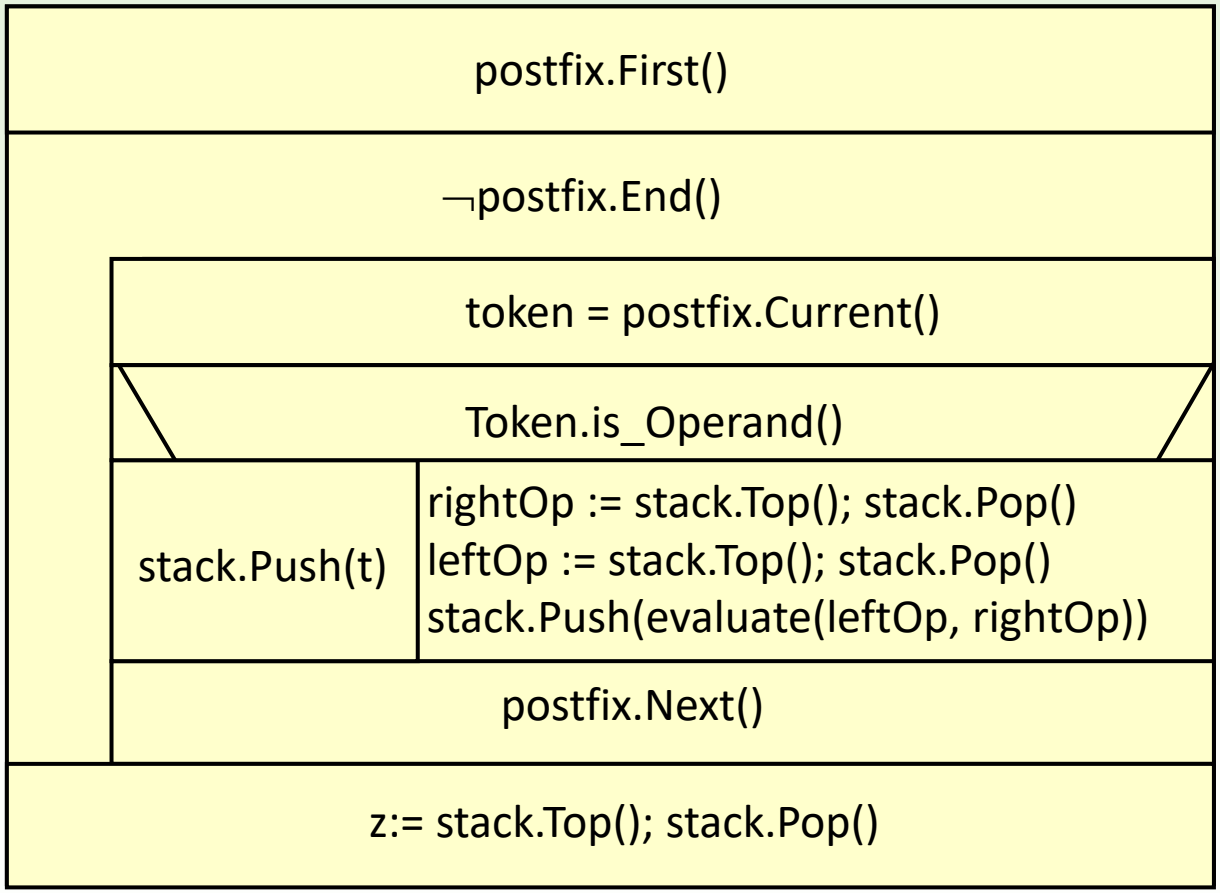
Postfix forma kiértékelése

A postfix forma operandusait (olvasásuk sorrendjében) egy verembe tesszük. Műveleti jel olvasása esetén a verem tetején levő két értéket (csak bináris műveleteink vannak) kivesszük, azokat a szóban forgó művelettel feldolgozzuk, és az eredményt visszatesszük a verembe. A feldolgozás végén a veremben találjuk kifejezés értékét.

5	11	26	+	43	4	-	*	+



Kiértékelés algoritmusa



Kiértékelés

```
// Evaluation
try{
        Stack<int> stackInt;
        for( Token* token : postfix ){
                if ( token->is_Operand() ){
                        stackInt.push( ((Operand*) token)->value() );
                } else{
                        int rightOp = stackInt.top(); stackInt.pop();
                        int leftOp  = stackInt.top(); stackInt.pop();
                        stackInt.push(((Operator*) token)->evaluate(leftOp, rightOp));
                }
        }
        int result = stackInt.top(); stackInt.pop();
        if( ! stackInt.empty() ){
                cout << "Syntax error!";
                throw Error;
        }
        cout << "value of the expression: " << result << endl;
}catch( StackExceptions ex ){
        if( ex==EMPTYSTACK ){
                cout << "Syntax error! ";
                throw Error;
        }
}
```

sablon osztály példányosítása

felsorolás

statikus konverzió

statikus konverzió

hiba lehetőség:
több operandus

hiba lehetőség:
kevés operandus

main.cpp

Tervezési minták I.
2.rész
Bináris fa és bejárása

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

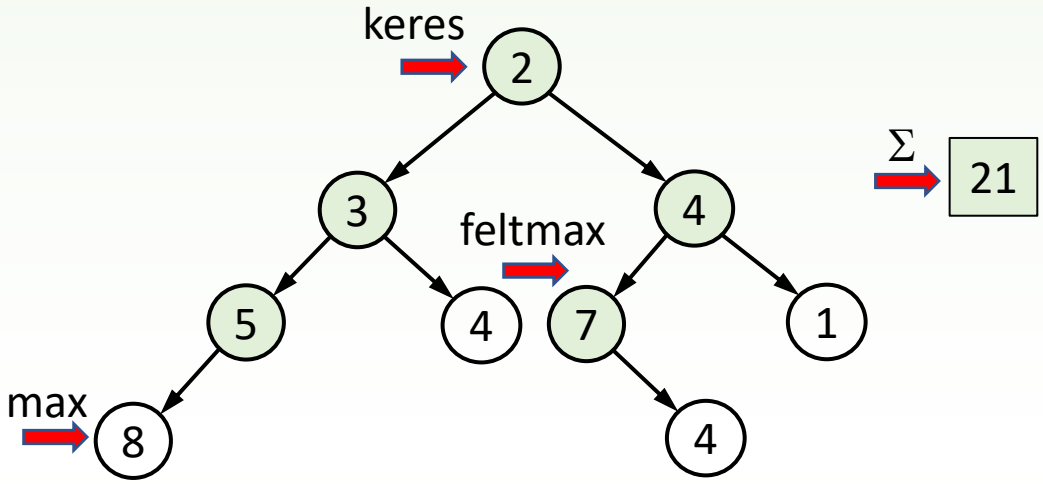
Feladat: Bináris fa bejárása

Építsünk fel megadott egész számokból egy bináris fát úgy, hogy véletlen módon dőljön el, hogy egy szám melyik csúcsba kerüljön.

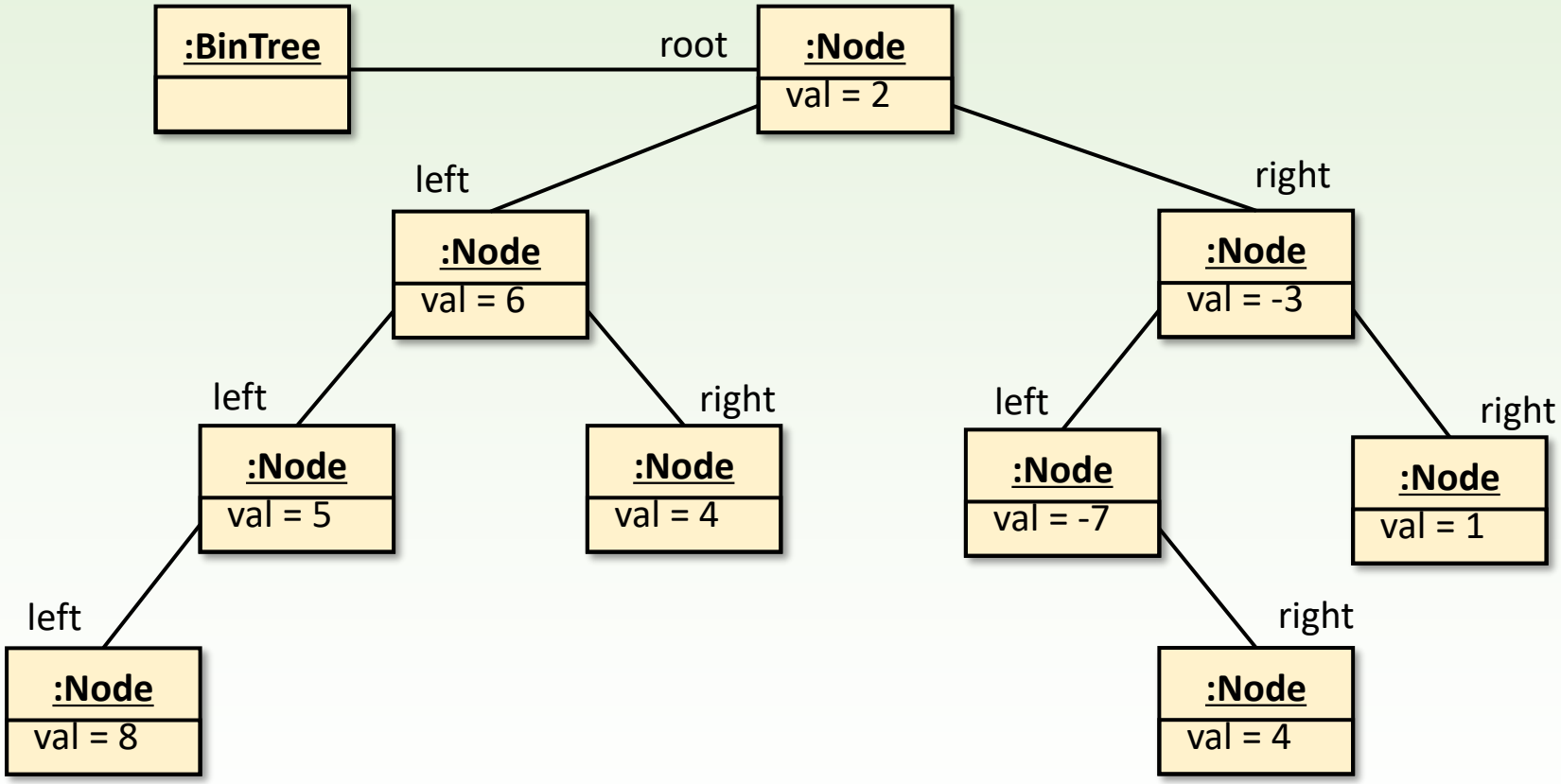
Írjuk ki a csúcsokban tárolt értékeket különféle bejárási stratégiák alapján.

Oldjunk meg olyan programozási feladatokat, mint például a belső csúcsok értékeinek összege, az összes csúcs vagy csak a belső csúcsok értékeinek maximuma, az „első” páros szám keresése valamelyik bejárási szerint.

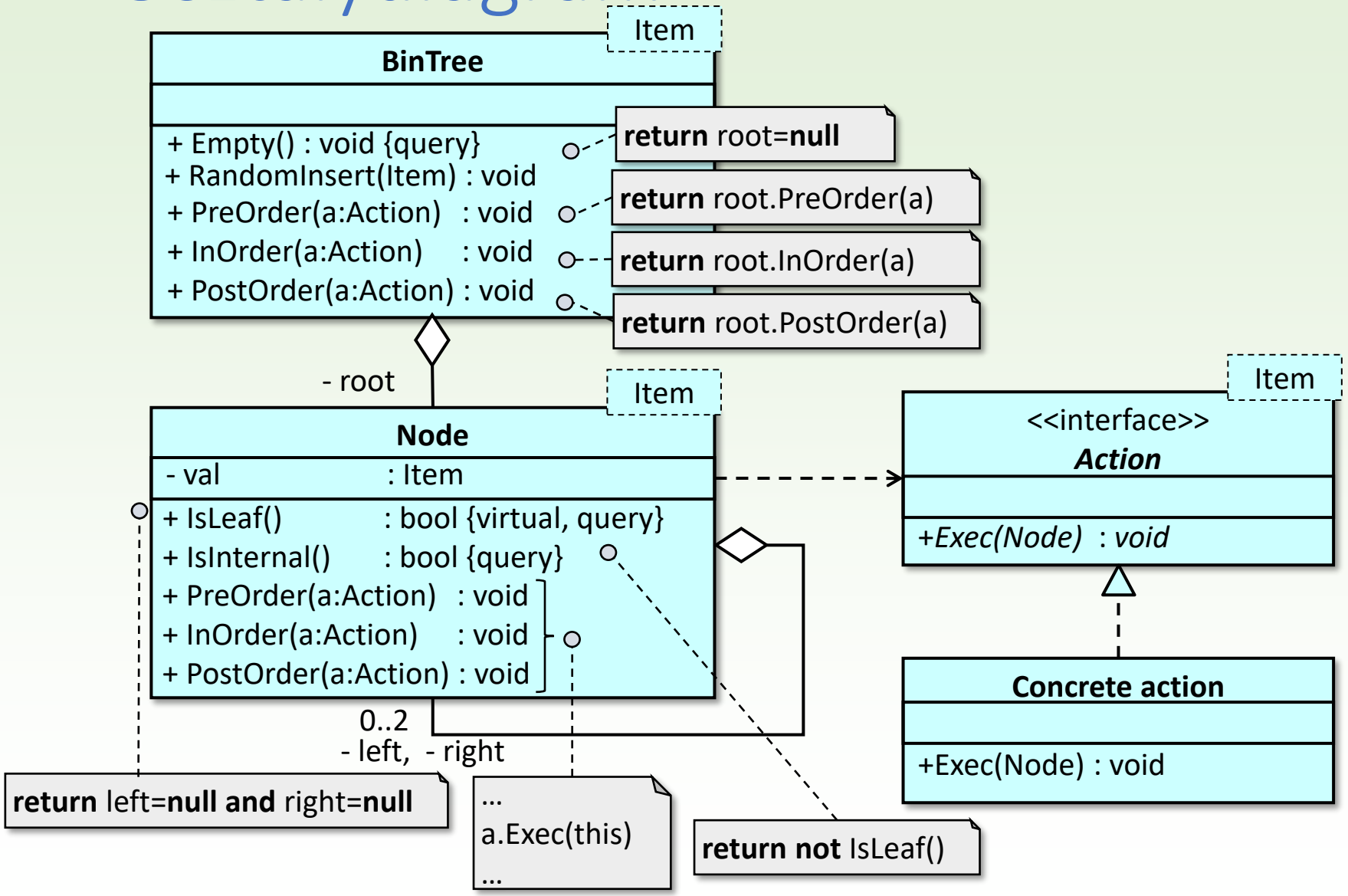
A bináris fát úgy tervezzük meg, hogy a csúcsaiban tárolt értékek típusa könnyen megváltoztatható legyen más olyan típusra, amelyen szintén értelmezhető összegzés, maximum kiválasztás, és keresés.



Bináris fa objektum diagramja

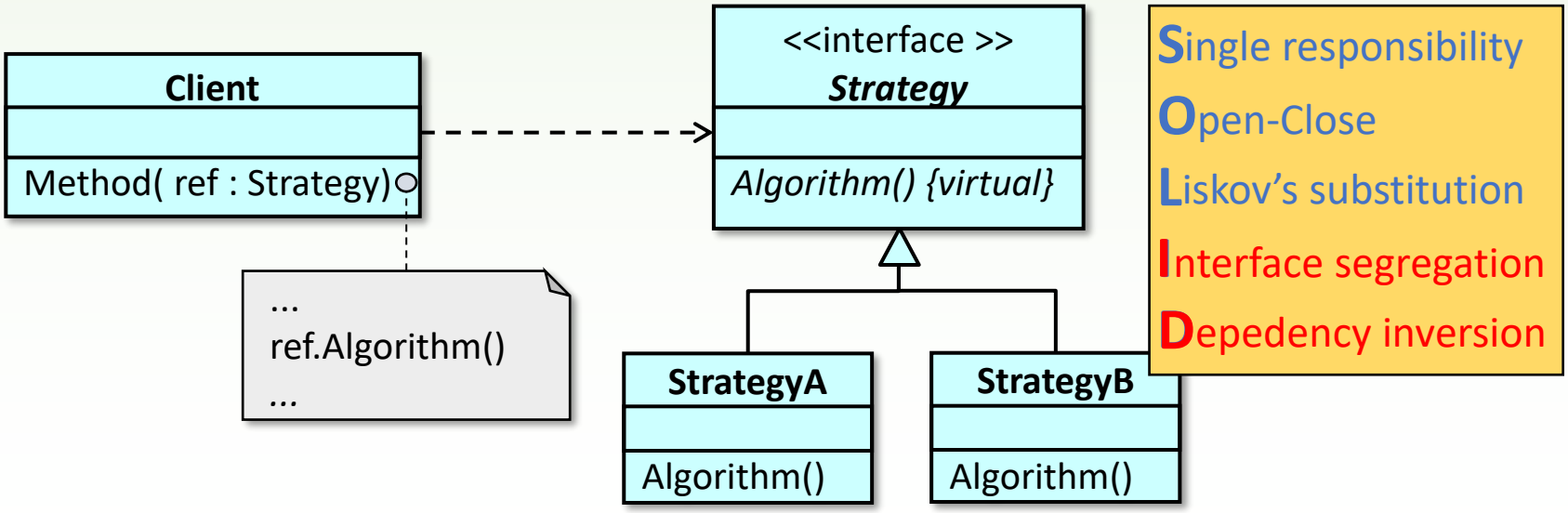


Osztálydiagram



Stratégia (strategy) tervezési minta

- Tevékenységek (algoritmusok) családját definiáljuk egy közös interfészt megvalósító osztályok metódusaként azért, hogy egy futási időben dőlhessen el az, hogy melyiket használjuk fel egy másik (kliens) osztály egyik metódusában. Egy-egy tevékenységet egy objektum hordoz a saját metódusaként, és ezt az objektumot kapja meg a kliens osztály metódusa paraméterként (vagy be is aggregálható egy kliens osztályú objektumba.



Tevékenység osztálysablonja



Itt az #include ”node.hpp” helyett
elég a Node definíciójának megelőlegezése,
hiszen mást nem használ fel a Node-tól.

```
#include ”node.hpp”

template <typename Item> class Node;

template <typename Item>
class Action{
public:
        virtual void exec(Node<Item> *node) = 0;
        virtual ~Action(){}
};
```

action.hpp

```
#include ”action.hpp”

template <typename Item>
class Node {
public:
        …
        void preOrder (Action<Item>*todo) { … todo->exec(this); … }
        void inOrder   (Action<Item>*todo) { … todo->exec(this); … }
        void postOrder(Action<Item>*todo) { … todo->exec(this); … }

private:
        …
};
```

node.hpp

Csúcs osztálysablonja

node.hpp

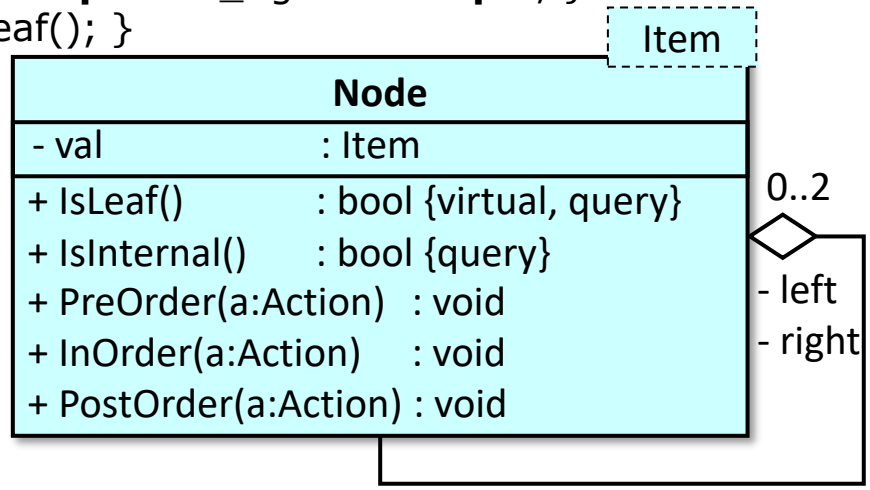
```
template <typename Item>
class Node {
public:
    Node(const Item& v): _val(v), _left(nullptr), _right(nullptr) {}

    Item getValue() const { return _val; }
    Node* getLeft() const { return _left; }
    Node* getRight() const { return _right; }
    void setLeft(Node* p)   {  _left = p; }
    void setRight(Node* p) {  _right = p; }

    bool isLeaf() const { return _left==nullptr && _right==nullptr; }
    bool isInternal() const { return !isLeaf(); }

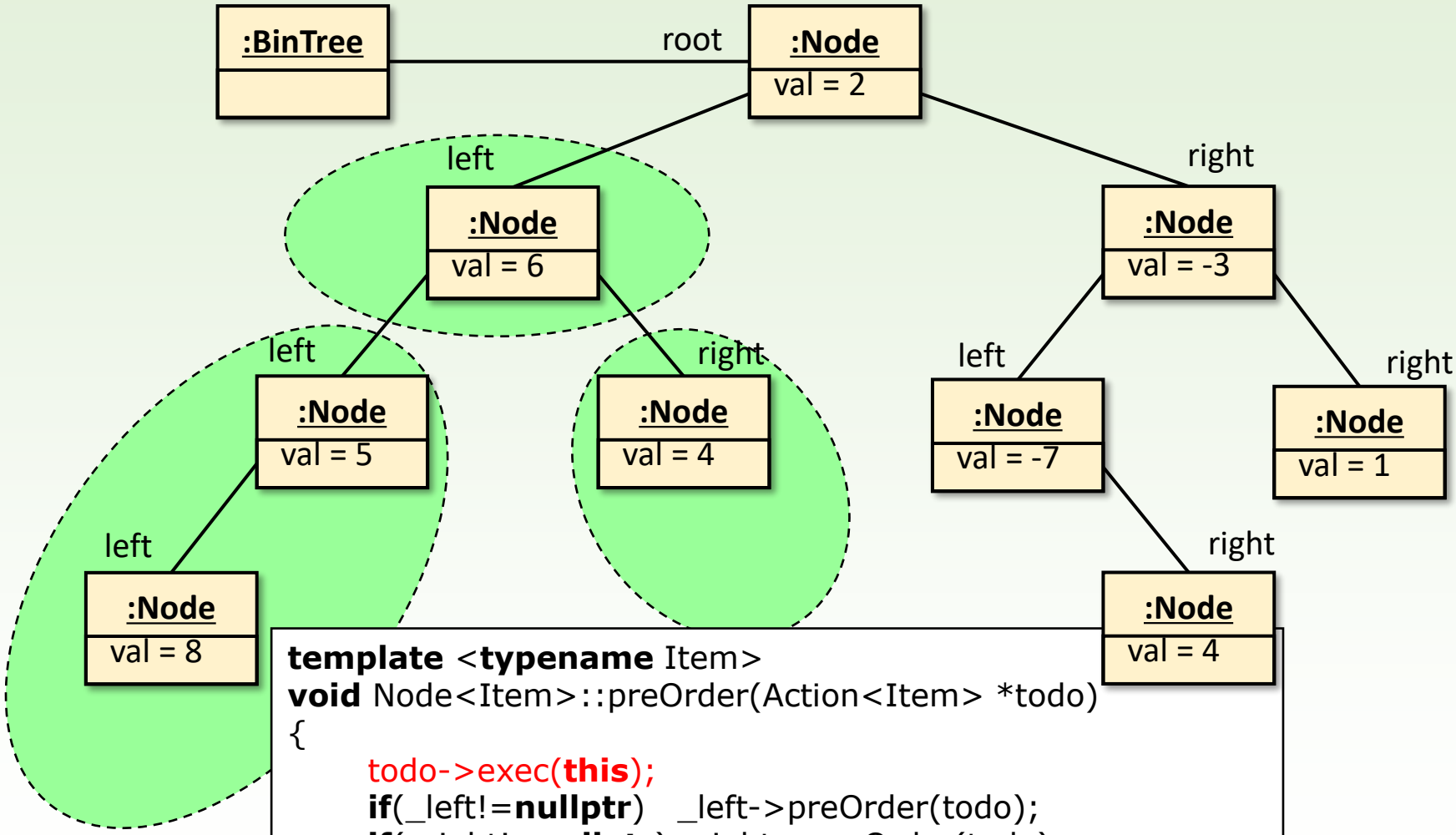
    void preOrder (Action<Item>*todo);
    void inOrder   (Action<Item>*todo);
    void postOrder(Action<Item>*todo);

private:
    Item _val;
    Node* _left;
    Node* _right;
};
```



preorder bejárás

2 6 5 8 4 -3 -7 4 1

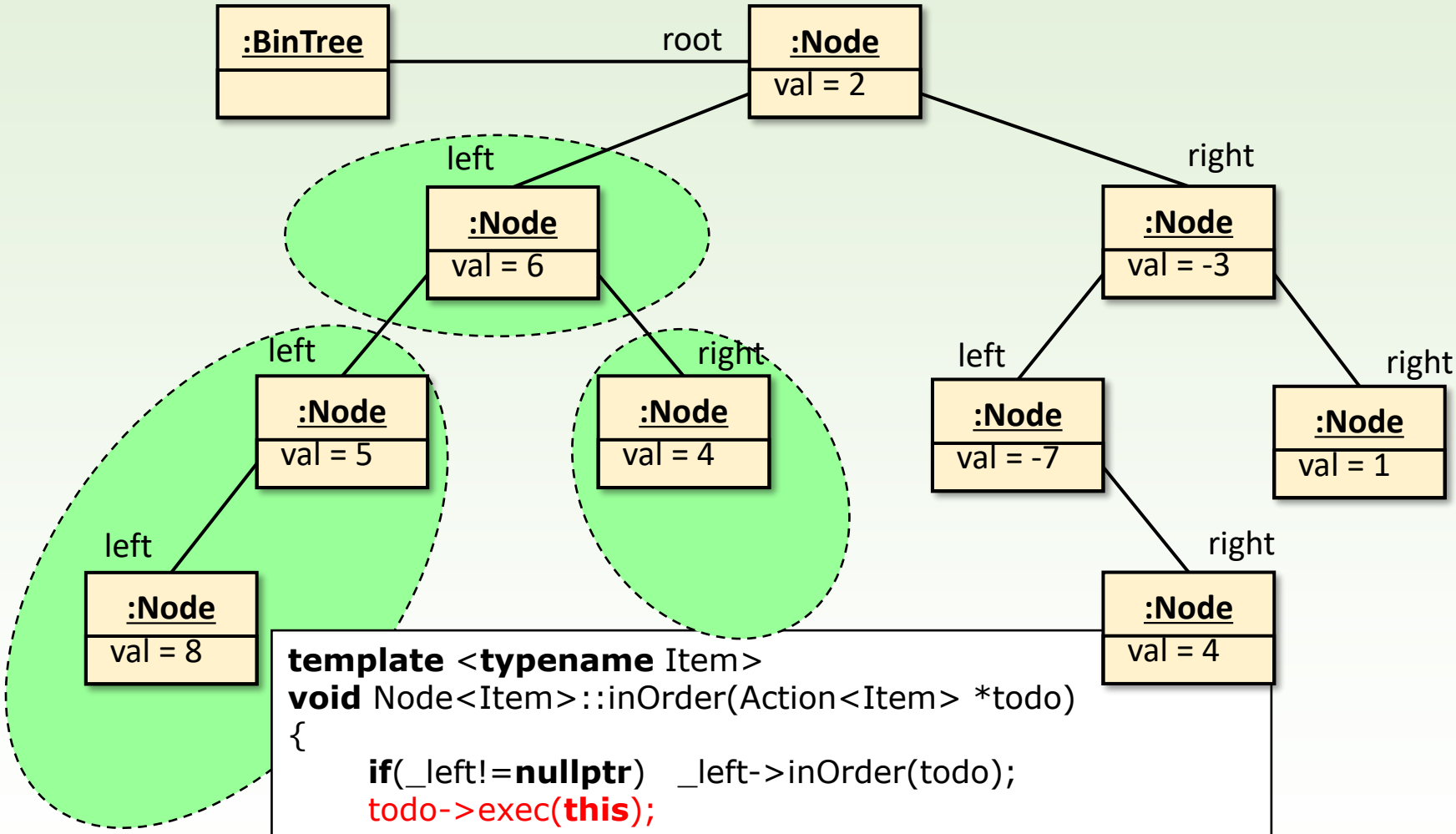



```
template <typename Item>
void Node<Item>::preOrder(Action<Item> *todo)
{
        todo->exec(this);
        if(_left!=nullptr)   _left->preOrder(todo);
        if(_right!=nullptr) _right->preOrder(todo);
}
```

bintree.hpp

Inorder bejárás

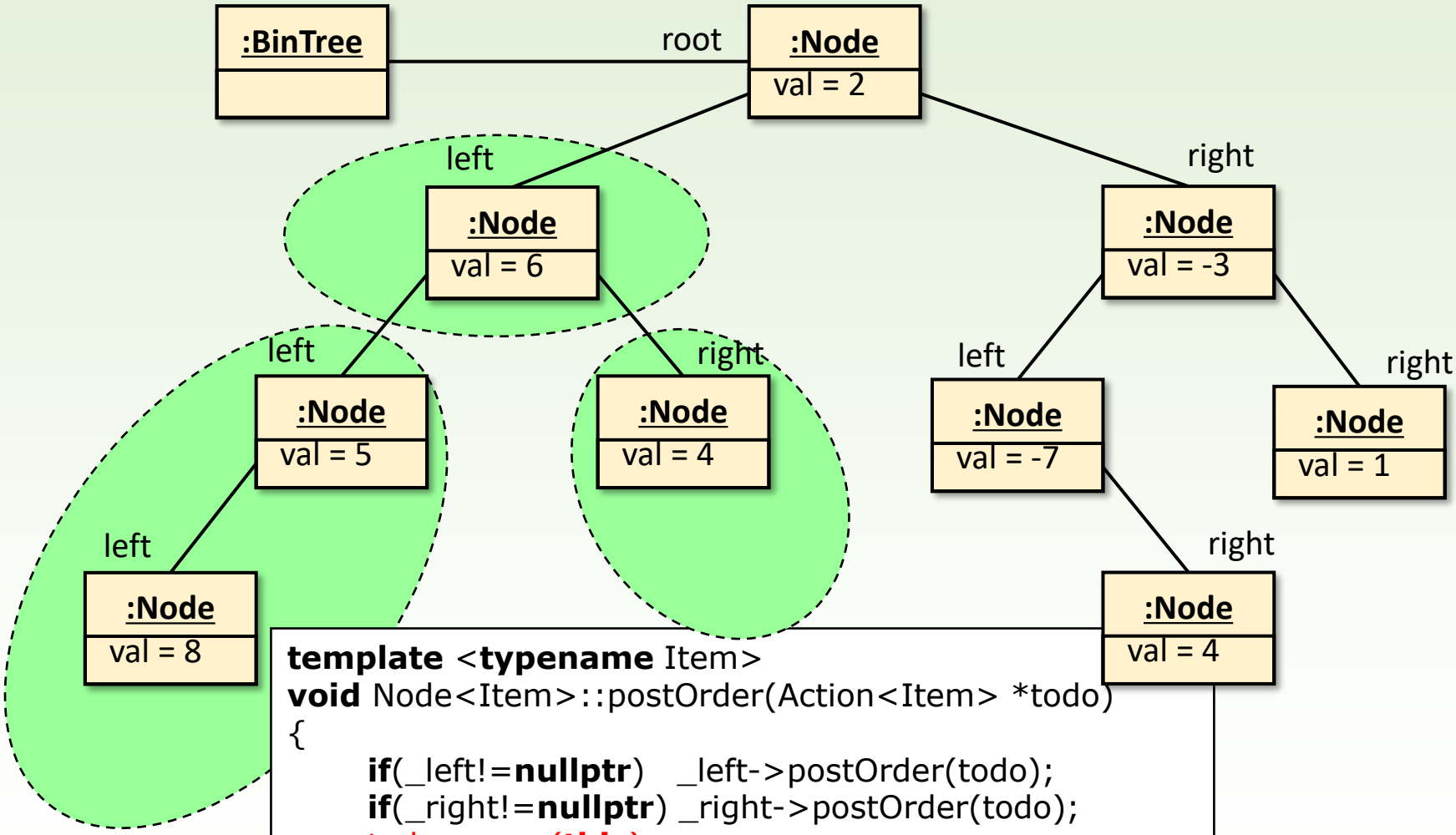
8 5 6 4 2 -7 4 -3 1




```
template <typename Item>
void Node<Item>::inOrder(Action<Item> *todo)
{
        if(_left!=nullptr)   _left->inOrder(todo);
        todo->exec(this);
        if(_right!=nullptr) _right->inOrder(todo);
}
```

bintree.hpp

postorder bejárás 8 5 4 6 4 -7 1 -3 2



Bináris fa osztálysablonja

véletlenszám-generátor inicializálása:
#include <time.h>
#include <cstdlib>

```
template <typename Item>
class BinTree{
public:
        BinTree():_root(nullptr) {srand(time(nullptr));}
        ~BinTree();

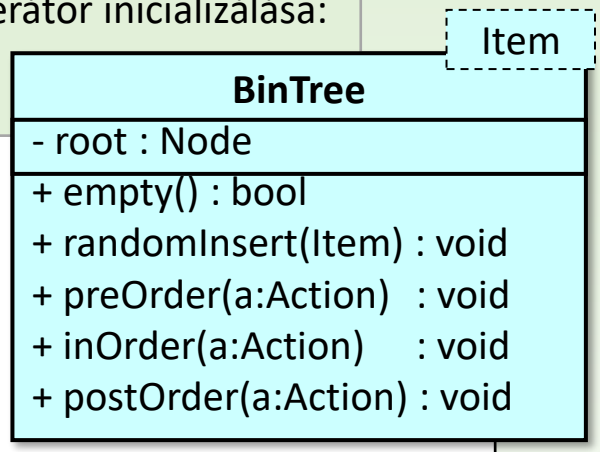
        enum Exceptions{NOROOT};
        Item rootValue() const {
                if( _root==nullptr ) throw NOROOT;
                return _root-> getValue();
        }

        bool empty() const { return null == _root; )
        void randomInsert(const Item& e);

        void preOrder (Action<Item>*todo){ _root->pre(todo); }
        void inOrder   (Action<Item>*todo){ _root->in(todo); }
        void postOrder(Action<Item>*todo){ _root->post(todo); }

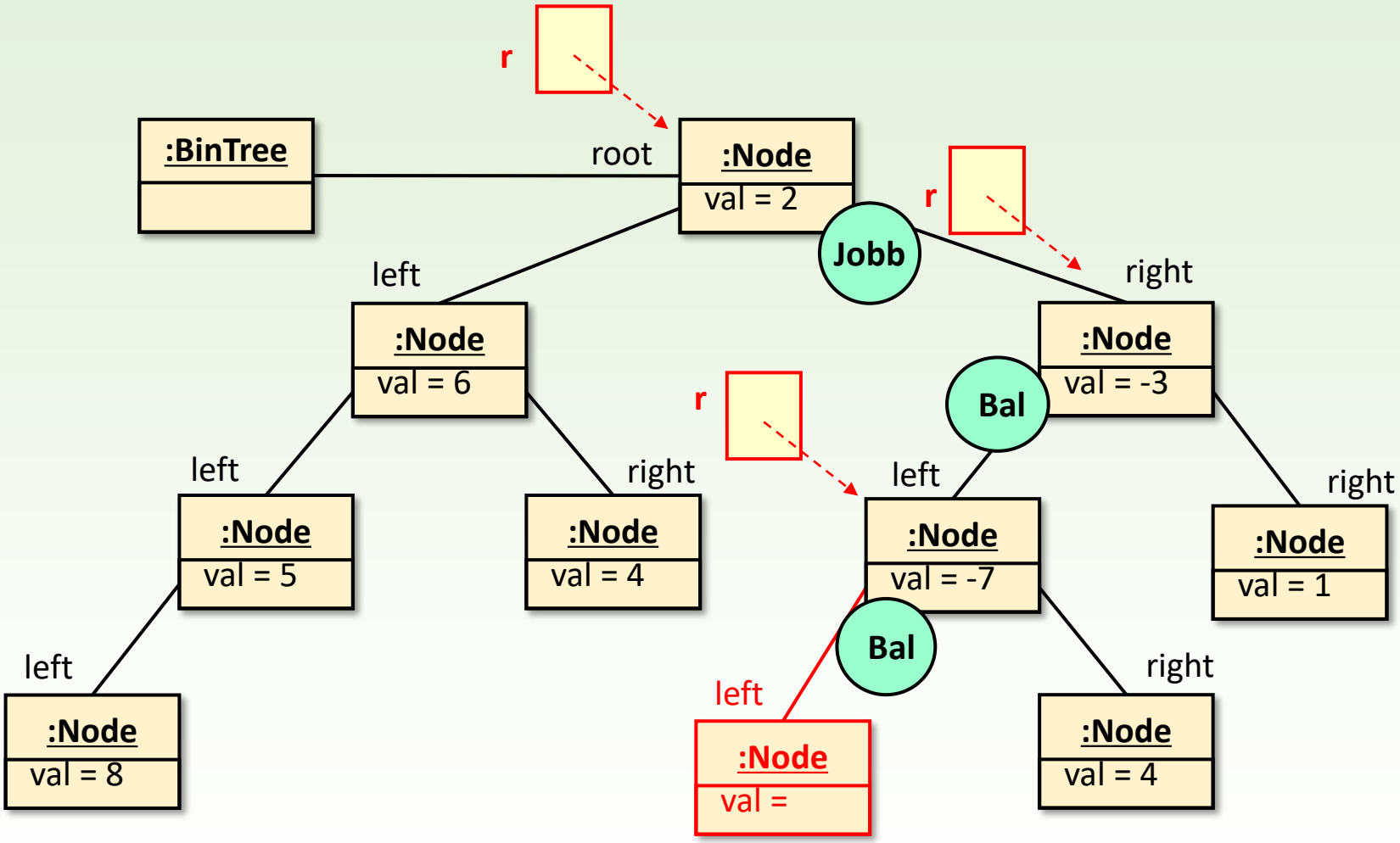
private:
        Node<Item>* _root;
};
```

Kivételt dob, ha a bináris fa üres, azaz még gyökércsúcsa sincs.



bintree.hpp

Új csúcs beszúrása a bináris fába



Új csúcs beszúrása a bináris fába

```
void BinTree<Item>::randomInsert(const Item& e)
{
      if(_root==nullptr) _root = new Node<Item>(e);
      else {
            Node<Item> *r = _root;
            int d = rand();
            while(d&1 ? r->getLeft()!=nullptr : r->getRight()!=nullptr){
                  if(d&1) r = r->getLeft();
                  else    r = r->getRight();
                  d = rand();
            }
            if(d&1) r->setLeft  ( new Node<Item>(e));
            else    r->setRight( new Node<Item>(e));
      }
}
```

véletlenszám generálása

A d szám utolsó bitje vajon 1-e?

bintree.hpp

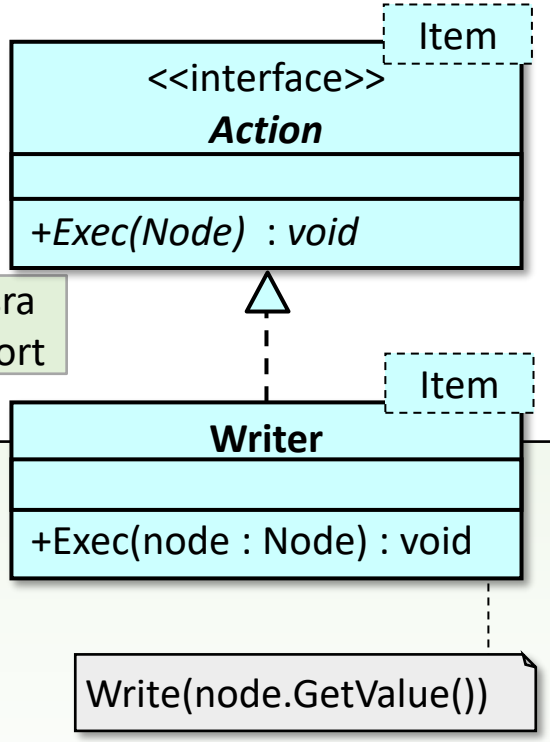
```
BinTree<int> t;
int i;
while(std::cin >> i) { t.randomInsert(i); }
```

int-ként nem értelmezhető inputra (pl. ‚a' betű) hamis értéket ad vissza

Kiírás tevékenység osztálysablonja

```
template <typename Item>
class Printer: public Action<Item>{
public:
        Printer(ostream &o): _os(o){};
        void exec(Node<Item> *node) override {
                _os << '[' << node->getValue() << ']';
        }
private:
        ostream& _os;
};
```

Item paraméter helyébe írt típusra legyen értelmezve a kiíró operátort



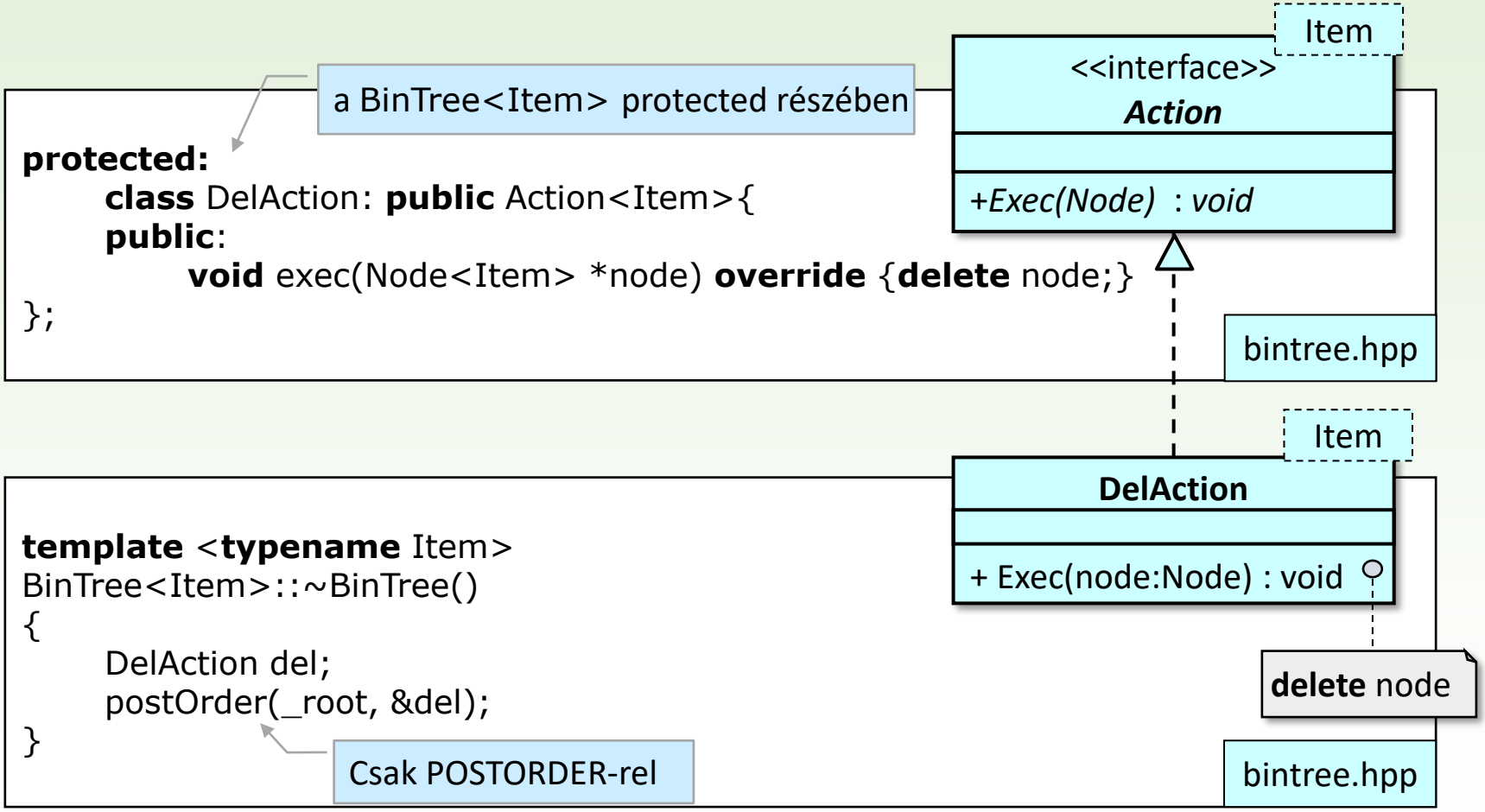

```
BinTree<int> t ;
…
Printer<int> print(cout);

cout << "preorder traversal :";
t.preOrder(&print);     cout << endl;

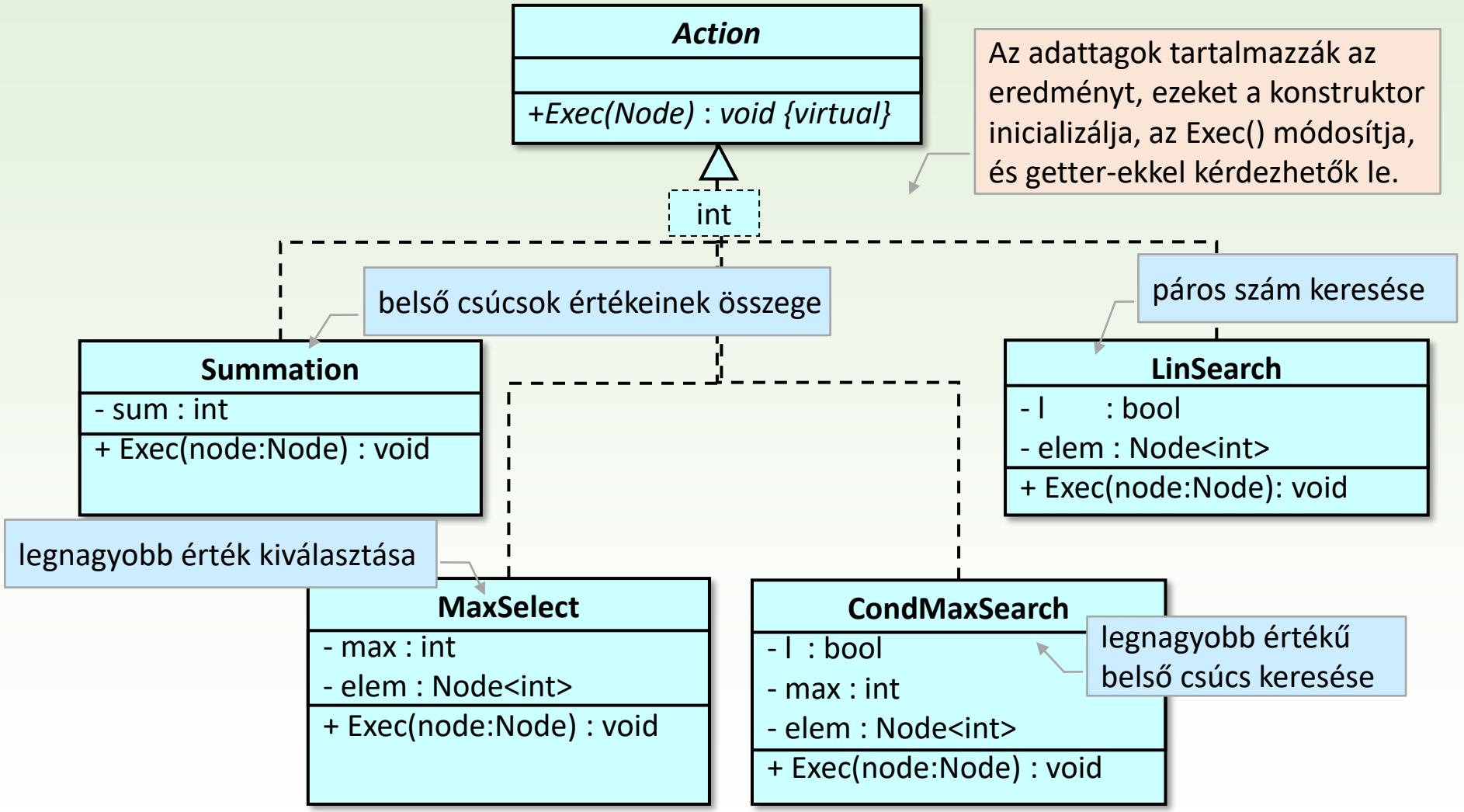
cout << "Inorder traversal :";
t.inOrder(&print);      cout << endl;

cout << "postorder traversal :";
t.postOrder(&print);    cout << endl;
```

Destruktor: bejárás törléssel



Egyéb tevékenység osztályok

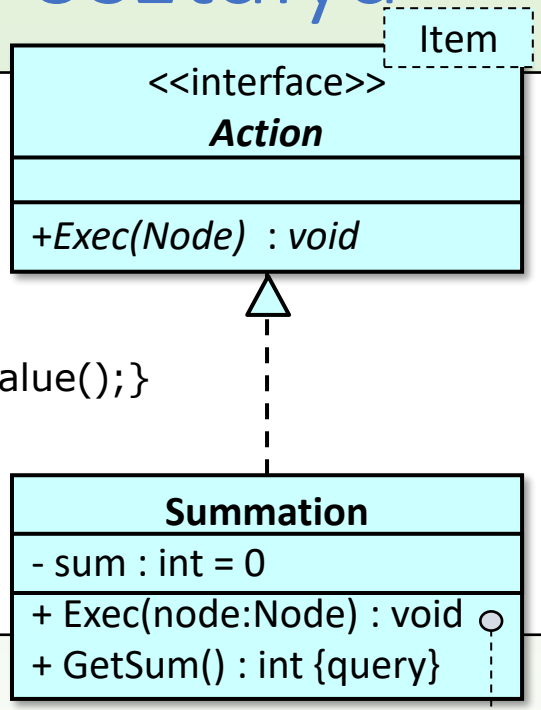


Összegzés tevékenység osztálya

```
class Summation: public Action<int>
{
public:
      Summation() : _sum(0) {}

      void exec(Node<int> *node) override {
            { if(node->isInternal()) _sum += node->getValue();}

      int getSum() const {return _sum;}
private:
      int _sum;
};
```




```
BinTree<int> t;
...
Summation action;
t.preOrder(&action);

cout << "Sum of the elements of the tree: " << action.getSum() << endl;
```

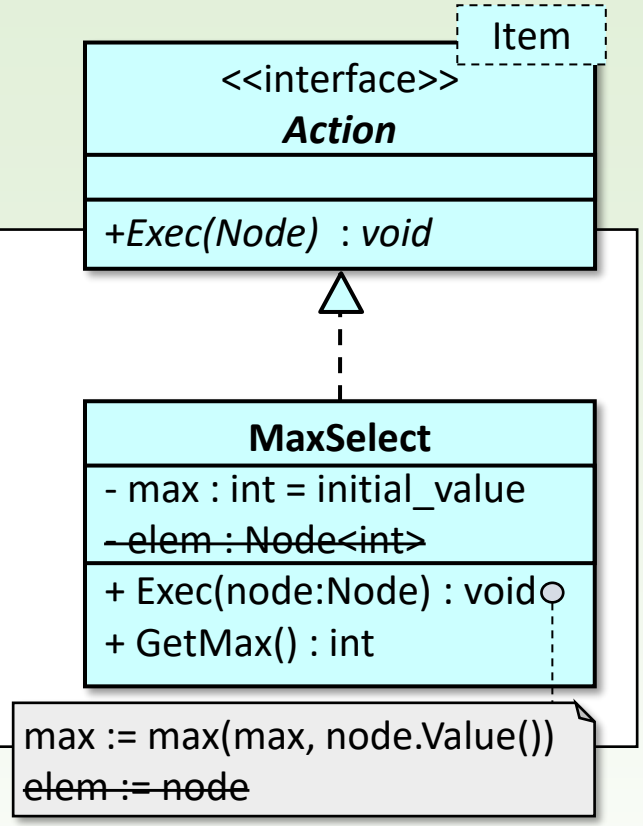
```
if node.IsInternal() then
    sum := sum + node.Value()
endif
```

Maximum kiválasztás

```
class MaxSelect: public Action<int>{
public:
      MaxSelect(int i) : _max(i){}

      void exec(Node<int> *node) override
        {_max = max( _max, node->getValue() ); }

      int getMax() const {return _max;}
private:
      int _max;
};
```

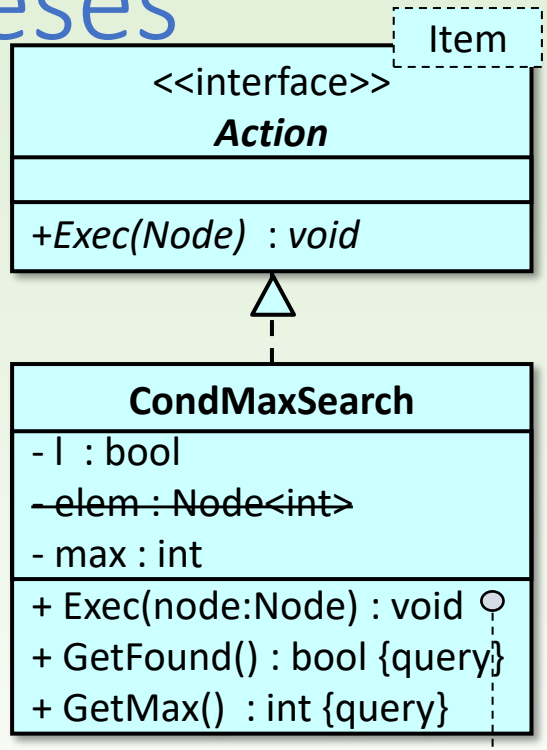



```
BinTree<int> t;
...
try {
   MaxSelect action(t.rootValue());
   t.preOrder(&action);
   cout << "Maxima of the elements of the tree: " << action.getMax() ;
}
```

kivételt dob, ha a bináris fa üres

Feltételes maximum keresés

```
class CondMaxSearch: public Action<int>{
public:
   CondMaxSearch(){_found = false;}
   virtual void exec(Node<int> *node)
   {
      if(node->isInternal()){
         if(!_found){
            _found = true;
            _max = node->getValue();
         }
         else  _max = std::max(_max, node->getValue());
      }
   }
   bool getFound() const { return _found;}
   int  getMax()     const { return _max; }
private:
   int _max;
   bool _found;
};
```

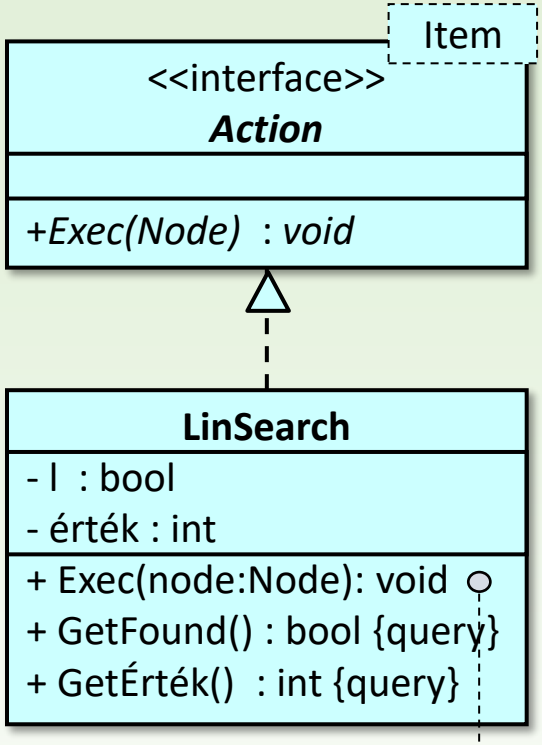



```
if node.IsLeaf() then return endif
if not l  then
    l, max := true, node.Value()
else  max := max(max, node.Value())
endif
```

```
BinTree<int> t;
...
CondMaxSearch action;
t.preOrder(&action);
cout << "Maxima of the elements of the tree: " << endl;
if(action.getFound()) cout << action.getMax() << endl;
else                  cout << "none" << endl;
```

Lineáris keresés

```
class Search : public Action<int>{
public:
    virtual void exec(Node<int> *node)
    {
        if(node->getValue()%2==0){
            _ value = node->getValue(); throw Found;
        }
    }
    enum Result{Found};
    int getValue(){return _ value;}
private:
    int _value;
};
```




```
BinTree<int> t;
...
Search action;
try{
        t.preOrder(&action);
        cout << endl << "No even numbers" << endl;
}catch(Search::Result e){
        if(e==Search::Found)
            cout << endl << "First even number: " << action.getValue() << endl;
}
```

```
if node.Value() mod 2 = 0 then
    l := true
    érték := node.Value()
endif
```